

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский Политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности

КУРСОВАЯ РАБОТА

**«Разработка синтаксического анализатора языка Bash»
по дисциплине «Формальные грамматики и теория компиляторов»**

Выполнил

Чупров Е.А.

студент гр. 5131001/30501

Руководитель

ст. преподаватель

Семьянов П.В.

Санкт-Петербург

2026

ОГЛАВЛЕНИЕ

1 ЦЕЛЬ РАБОТЫ	3
2 ХОД РАБОТЫ	4
2.1 Теоретические сведения	4
2.2 Лексический анализатор языка	5
2.3 Синтаксический анализатор языка	7
2.4 Возникшие конфликты и их исправление	13
2.5 Работоспособность парсера	15
3 РЕЗУЛЬТАТЫ	21
4 ВЫВОД	22
ПРИЛОЖЕНИЕ 1	23
ПРИЛОЖЕНИЕ 2	26

1 ЦЕЛЬ РАБОТЫ

Получение навыков проектирования грамматик языков программирования, самостоятельная разработка контекстно-свободной грамматики подмножества языка Bash и создание его синтаксического распознавателя с использованием генераторов лексических и синтаксических анализаторов flex и yacc.

Распознаватель решает только вопрос принадлежности текста языку. Сценарий он не исполняет и подстановки не раскрывает.

2 ХОД РАБОТЫ

2.1 Теоретические сведения

Bash — командный интерпретатор GNU и язык сценариев UNIX. По синтаксису он продолжает Bourne shell и раздел Shell Command Language стандарта POSIX. Свои расширения у Bash есть поверх этой основы. Сценарий — это последовательность команд.

Весь Bash в одну контекстно-свободную грамматику не помещается. Знаки #, \$, скобки и перевод строки в кавычках, в here-документе и в арифметике читаются по-разному. Алиасы, тильда и раскрытие {a,b} меняют текст ещё до разбора, поэтому в грамматику они не входят.

В работе взято подмножество языка. Файл начинается со строки #!/bin/bash. После пути можно оставить только пробелы и табы. Файл, состоящий только из шебанга, принимается. Пустой файл не принимается. Дальше идут обычные команды с аргументами и перенаправлениями >, >>, <, 2>, 2>&1 в конце. Присваивание NAME=value считается отдельной командой. Команды соединяются конвейером, &&, ||, ;, & и переводом строки.

Проверка пишется как [...]. Знаки вроде -lt остаются обычными словами. Есть if с elif и else, циклы while, until, for имя in список и case. Каждая ветка case заканчивается ;;. В условии if, while и until разрешена только проверка [...]: любая команда на этом месте не даёт ясной границы между условием и телом. Функция записывается как name() { ...; }, function name { ...; } или function name() { ...; }. Фигурные скобки здесь только тело функции, перед } нужны ; или перевод строки. У local указываются имя, a= или a=1. Вне кавычек допускаются \$имя, \$1, \$#, \$@, \$*, \$\$, \$?, \${имя} и \$(...). Внутри \$(...) тот же список команд, но без шебанга. В кавычках \$ не разбирается.

Не вошло то, чему нужен отдельный режим сканера или своя грамматика. Here-документ читает текст до слова-ограничителя, поэтому << отвергается. `[ [ ... ] ]`, `(( ... ))` и `$(( ... ))` — это отдельные выражения, и проверка оставлена только в виде `[ ... ]`. В `${...}` можно написать лишь имя: без `${var:-default}`, массивов и обратных кавычек. Группа `{ ...; }`, подшелл `( ... )` и `<( ... )` занимают те же скобки, что функция и `$(...)`, и в язык не входят. Запись `VAR=value command` тоже не входит. `FOO=1` — присваивание, `echo FOO=1` — команда с аргументом. Не принимаются шебанг через `/usr/bin/env`, аргументы после пути.

Часть отказов идёт от самих лексем. Слова `done` и `in` везде ключевые, поэтому `echo done` — ошибка. Вне кавычек нельзя поставить `\`. Скобки `[ ] { }` в слово не входят, так что `ls [abc]*` и `echo {a,b}` не принимаются. Не принимаются также `if ! [ ... ]` и перенаправление после `done`.

2.2 Лексический анализатор языка

Лексический анализатор разбивает входной текст сценария на последовательность лексем. Сканер начинает в состоянии INITIAL и переходит в состояние BODY только после шебанга `#!/bin/bash`. Любой другой первый символ, лишний текст в строке шебанга и пустой файл дают ошибку на строке 1. Комментарии `#!.*` обрабатываются только внутри состояния BODY.

Основные типы токенов и регулярные выражения приведены в таблице 1. Макросы flex записаны так же, как в `lex_bash.l`.

Категория	Конструкции и токены
Шебанг и комментарии	<code>#!/bin/bash"[\t]*\n, "#!/bin/bash"[\t]*, #.*</code>
Ключевые слова	<code>if, then, elif, else, fi, while, until, do, done, for, in, case, esac, function, local</code>
Имена, слова, присваивание	ID — <code>[A-Za-z_][A-Za-z0-9_]*</code> . WORD — непробельная последовательность без операторов, кавычек и \$ (точный класс записан в приложении 1). <code>{ID}={WORD}</code> и <code>{ID}=</code> возвращают ASSIGN
Переменные подстановки и	<code>"\${ID}" (DOLLAR_ID), "\$"[0-9]+ (DOLLAR_NUM), \$# \$@ \$* \$\$ \$? (DOLLAR_SPECIAL), "\${ID}" (DOLLAR_BRACE), "\$(" (DOLLAR_LPAREN)</code>
Строковые литералы	' и ' читаются функциями и возвращают STRING
Операторы и разделители	<code>&&, , ;;, :, &, , <, >, >>, 2>, 2>&1, (,), {, }, [,],</code> перевод строки (NL)

Таблица 1 – Ключевые конструкции лексического анализатора

Длинные операторы стоят в сканере раньше коротких, поэтому `>>` не распадается на два `>`, а `2>&1` — на `2>` и отдельный фрагмент. Пробел и таб токенов не порождают. Перевод строки в теле программы — токен NL. При включённом `%option yylineno` flex увеличивает номер строки в момент чтения этого символа.

Полный код анализатора приведён в **Приложении 1**.

2.3 Синтаксический анализатор языка

Элемент	Описание
program	Вся программа: шебанг и список команд
list	Список команд с необязательными пустыми строками
stmt_seq	Последовательность конструкций and_or
separator	Разделитель команд: ;, & или перевод строки
nl_opt	Ноль или более переводов строки
and_or	Конвейеры, связанные через && /
pipeline	Команды, связанные через
command	Простая команда, присваивание, [...] или составная конструкция
simple_command	Имя, аргументы и редиректы в конце
assignment	Присваивание NAME= / NAME=value
test_command	Проверка [...]
if_clause, while_clause, until_clause, for_clause, case_clause	Составные команды
function_def, brace_group, local_stmt	Функция, её тело и local
cmdsubst	Подстановка \$(list)
sep	Разделитель перед then / do: ; или перевод строки

Таблица 2 – Основные синтаксические элементы

Стартовый символ — program. Иерархия правил сверху вниз:
program → list → stmt_seq → and_or → pipeline → command. В правилах, которые описывают допустимый язык, семантических действий нет: анализатор не вычисляет значения команд.

```

38  program:
39  |      SHEBANG list
40  |  |  ;
41
42  list:
43  |  |  nl_opt
44  |  |  | nl_opt stmt_seq
45  |  |  | nl_opt stmt_seq separator
46  |  |  ;
47
48  stmt_seq:
49  |      and_or
50  |  |  stmt_seq separator and_or
51  |  |  ;
52
53  separator:
54  |      SEMI nl_opt
55  |  |  AMP nl_opt
56  |  |  NL nl_opt
57  |  |  ;
58

```

Рисунок 1 – Начало грамматики

Программа обязана начинаться с шебанга. За ним идёт список команд. Файл из одного шебанга принадлежит языку, пустой файл — нет.

`separator` завершает команду: `;` и перевод строки равносильны, `&` помечает фон. `nl_opt` разрешает пустые строки и перенос `&&`, `||`, `|` без пустой команды. Дальше грамматика сужается до одной команды.

```

64  and_or:
65      | pipeline
66      | and_or AND nl_opt pipeline
67      | and_or OR nl_opt pipeline
68      ;
69
70  pipeline:
71      | command
72      | pipeline PIPE nl_opt command
73      ;
74
75  command:
76      | simple_command
77      | assignment
78      | test_command
79      | if_clause
80      | while_clause
81      | until_clause
82      | for_clause
83      | case_clause
84      | function_def
85      | local_stmt
86      ;

```

Рисунок 2 – Фрагмент грамматики: списки `&&` и `||`, конвейер и команда

`and_or` — это цепочка конвейеров через `&&` и `||`. `pipeline` — цепочка команд через `|`. После `&&`, `||` и `|` может быть пустая строка: её забирает `nl_opt`. Конвейер стоит ниже списка, поэтому в записи `a && b | c` сначала связывается `b | c`, и только потом результат соединяется с `a` через `&&`. Одно звено такой цепочки — `command`: простая команда, присваивание, проверка `[ ... ]`, `if`, `while`, `until`, `for`, `case`, функция или `local`.

```

88  assignment:
89      |    ASSIGN
90      |    | ASSIGN word
91      |    ;
92
93  simple_command:
94      |    cmd_name arg_list redir_list
95      |    ;
96
97  cmd_name:
98      |    ID
99      |    | WORD
100     |    ;
101
102  arg_list:
103     |    /* empty */
104     |    | arg_list arg
105     |    ;
106
107  arg:
108     |    word
109     |    | ASSIGN
110     |    ;

```

Рисунок 3 – Присваивание и простая команда

Присваивание — это NAME= или NAME= и следующее слово, например FOO= и FOO=1. Простая команда состоит из имени, аргументов и перенаправлений в конце. Аргументом может быть и присваивание, поэтому echo FOO=1 остаётся обычной командой. Запись FOO=1 echo x сюда не входит.

```

112 word:
113     ID
114     | WORD
115     | STRING
116     | expansion
117     ;
118
119 expansion:
120     DOLLAR_ID
121     | DOLLAR_NUM
122     | DOLLAR_SPECIAL
123     | DOLLAR_BRACE
124     | cmdsubst
125     ;
126
127 cmdsubst:
128     DOLLAR_LPAREN list RPAREN
129     ;
130

```

Рисунок 4 – Слово и подстановки

Слово — это имя, обычный набор символов, строка в кавычках или подстановка. Подстановка бывает такой: \$имя, \$1, \$#, \$@, \$*, \$\$, \$?, \${имя} и \$(...). Внутри \$(...) разбирается тот же список команд, что и в теле файла, только без шебанга.

```

131  redir_list:
132      /* empty */
133      | redir_list redir
134      ;
135
136  redir:
137      GT redir_target
138      | DGREAT redir_target
139      | LT redir_target
140      | LT LT {
141          error_line(yylineno, "here-document is not supported");
142          exit(1);
143      }
144      | REDIR_ERR redir_target
145      | REDIR_ERR_OUT
146      ;
147
148  redir_target:
149      ID
150      | WORD
151      ;
152

```

Рисунок 5 – Перенаправления

Перенаправлений в конце команды может не быть. Допустимы `>`, `>>`, `<`, `2>` и `2>&1`. Цель `>`, `>>`, `<` и `2>` — имя или обычное слово. Два знака `<` подряд — это here-документ. Такое правило отвергает его и печатает `here-document is not supported`.

```

153 test_command:
154     LBRACK arg_list RBRACK
155     ;
156
157 if_clause:
158     IF test_command sep THEN list FI
159     | IF test_command sep THEN list elif_parts FI
160     | IF test_command sep THEN list ELSE list FI
161     | IF test_command sep THEN list elif_parts ELSE list FI
162     | IF bad_condition {
163         error_line(yylineno, "if condition must be [ ... ]");
164         exit(1);
165     }
166     ;
167
168 elif_parts:
169     ELIF test_command sep THEN list
170     | elif_parts ELIF test_command sep THEN list
171     ;
172
173 while_clause:
174     WHILE test_command sep DO list DONE
175     | WHILE bad_condition {
176         error_line(yylineno, "while condition must be [ ... ]");
177         exit(1);
178     }
179     ;
180
181 until_clause:
182     UNTIL test_command sep DO list DONE
183     | UNTIL bad_condition {
184         error_line(yylineno, "until condition must be [ ... ]");
185         exit(1);
186     }
187     ;
188
189 /* Условие if/while/until без '[': одно слово и разделитель. */
190 bad_condition:
191     word SEMI
192     | word NL
193     ;

```

Рисунок 6 – Условие if, while и until

Проверка [...] — это слова в квадратных скобках. В if после неё идут then и список команд, затем необязательные elif и else, и в конце fi. Перед then и do нужны ; или перевод строки. Циклы while и until устроены так же, только тело стоит между do и done. Если вместо [...] написано одно слово, как в if true;, срабатывает bad_condition и печатается сообщение, что условие должно быть [...].

```

217 patterns:
218     pattern
219     | patterns PIPE pattern
220     ;
221
222 pattern:
223     ID
224     | WORD
225     | STRING
226     ;
227
228 function_def:
229     ID LPAREN RPAREN nl_opt brace_group
230     | FUNCTION ID nl_opt brace_group
231     | FUNCTION ID LPAREN RPAREN nl_opt brace_group
232     ;
233
234 brace_group:
235     LBACE nl_opt RBACE
236     | LBACE nl_opt stmt_seq separator RBACE
237     ;
238

```

Рисунок 7 – Образцы case, функция и её тело

Образец в case — это имя, слово или строка. Несколько образцов в одной ветке пишутся через |, как в a|b). Функция задаётся тремя способами: name() { ...; }, function name { ...; } и function name() { ...; }. Тело в фигурных скобках может быть пустым. Если внутри есть команды, перед } нужны ; или перевод строки. Запись name() { echo x } без разделителя не принимается.

```

238
239 local_stmt:
240 |    LOCAL local_names
241 |    ;
242
243 local_names:
244 |    local_item
245 |    | local_names local_item
246 |    ;
247
248 local_item:
249 |    ID
250 |    | ASSIGN
251 |    | ASSIGN word
252 |    ;
253

```

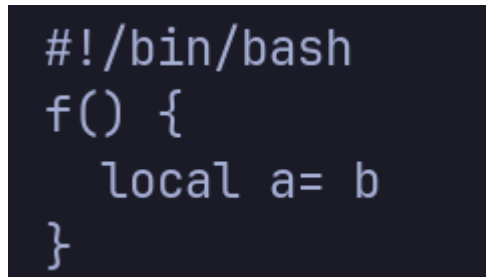
Рисунок 8 – Объявление local

У local должен быть хотя бы один аргумент. Это имя (a), присваивание без значения (a=) или со значением (a=1). Несколько аргументов пишутся подряд: local a b=1. Внутри функции стоит объявление или нет, грамматика не проверяет.

2.4 Возникшие конфликты и их исправление

Команда `bison -d -y -v pars_bash.y` сообщает один конфликт сдвига и свёртки: для одной лексемы можно и прочитать её дальше, и закончить уже начатое правило. Конфликтов двух свёрток нет.

Один конфликт в `local`. Аргументы показаны на рисунке 8: имя, `a=` или `a=` со значением. Спорный текст — на рисунке 9: между `a=` и `b` стоит пробел.



```
#!/bin/bash
f() {
    local a= b
}
```

Рисунок 9 – Код, на котором виден конфликт в `local`

Сканер пробел выбрасывает, и парсер видит `ASSIGN (a=)` и следом `ID (b)`. Дальше два хода.

Сдвиг читает `b` как значение по правилу `ASSIGN word`: один аргумент.

Свёртка закрывает пустое `a=` по правилу `ASSIGN` и берёт `b` как следующее имя: два аргумента.

Оба хода этот текст принимают. Bison в таком споре всегда выбирает сдвиг, поэтому программа отвечает одинаково и править грамматику незачем. Без пробела, в `local a=b`, сканер отдаёт одну лексему `ASSIGN`, и спора нет.

На рисунке 10 сверху сборка без `-Wcounterexamples`, снизу тот же конфликт с этим флагом.

```

compiler_labs/course main • ? › ./compil.sh
lex
pars_bash.y: warning: 1 shift/reduce conflict [-Wconflicts-sr]
pars_bash.y: note: rerun with option '-Wcounterexamples' to generate conflict counterexamples
bison
gcc

compiler_labs/course main • ? › bison -d -y -v -Wcounterexamples pars_bash.y
pars_bash.y: warning: 1 shift/reduce conflict [-Wconflicts-sr]
pars_bash.y: warning: shift/reduce conflict on token ID [-Wcounterexamples]
Example: ASSIGN • ID
Shift derivation
  local_names
  ↳ 88: local_item
      ↳ 92: ASSIGN word
          ↳ 36: • ID
Reduce derivation
  local_names
  ↳ 89: local_names      local_item
      ↳ 88: local_item  ↳ 90: ID
          ↳ 91: ASSIGN •

```

Рисунок 10 – Конфликт сдвига и свёртки в local на токене ID

Сдвиг идёт по правилу ASSIGN word, свёртка — по правилу ASSIGN.

2.5 Работоспособность парсера

Если текст принадлежит подмножеству, программа ничего не печатает и завершается с кодом 0. Если найдена ошибка, печатается Error: line N и код возврата равен 1. N — номер строки, считая с 1. После первой ошибки разбор останавливается.

Для нескольких типичных случаев к номеру добавляется короткое пояснение: Error: line N: text. Остальные ошибки сообщаются только номером строки. На рисунке 15 ветка if закрыта словом done, поэтому в сообщении один номер строки.

На рисунках 11 и 12 два положительных теста. Сообщений нет, код возврата равен 0.



```
compiler_labs/course main • ? › cat tests/accept/12-if-elif-else.sh
#!/bin/bash
if [ "$1" = a ]; then
    echo a
elif [ "$1" = b ]; then
    echo b
else
    echo other
fi

compiler_labs/course main • ? › ./bash_parser tests/accept/12-if-elif-else.sh

compiler_labs/course main • ? ›
```

Рисунок 11 – Принимаемый сценарий: if, elif и else

```

compiler_labs/course main • ? > cat tests/accept/19-local.sh
#!/bin/bash
f() {
    local name
    local name="$1"
    local a b=1
}

compiler_labs/course main • ? > ./bash_parser tests/accept/19-local.sh

compiler_labs/course main • ? > █

```

Рисунок 12 – Принимаемый сценарий: функция и local

На рисунке 13 файл без шебанга: Error: line 1: missing shebang, код возврата равен 1. На рисунке 14 here-документ: Error: line 2: here-document is not supported, код возврата равен 1.

```

compiler_labs/course main • ? > cat tests/reject/no-shebang.sh
echo hello

compiler_labs/course main • ? > ./bash_parser tests/reject/no-shebang.sh
Error: line 1: missing shebang

compiler_labs/course main • ? × █

```

Рисунок 13 – Отказ: нет шебанга

```

compiler_labs/course main • ? × cat tests/reject/here-doc.sh
#!/bin/bash
cat <<EOF
x
EOF

compiler_labs/course main • ? > ./bash_parser tests/reject/here-doc.sh
Error: line 2: here-document is not supported

compiler_labs/course main • ? × █

```

Рисунок 14 – Отказ: here-документ не поддерживается

На рисунке 15 обычная синтаксическая ошибка: Error: line 4, код возврата равен 1.

```
compiler_labs/course main • ? › cat tests/reject/if-wrong-close.sh
#!/bin/bash
if [ a ]; then
    echo x
done

compiler_labs/course main • ? › ./bash_parser tests/reject/if-wrong-close.sh
Error: line 4
```

Рисунок 15 – Отказ: ошибка на строке 4

```
compiler_labs/course main • ? › cat ./tests/accept/30-overview.sh
#!/bin/bash
name="World"
count=0
if [ $# -lt 1 ]; then echo usage; fi
echo $name | wc -l > /tmp/out
greet() {
    local name="$1"
    echo ${name}
}
for item in a b; do greet $item; done
case $item in
a|b) echo ok ;;
*) echo other ;;
esac

compiler_labs/course main • ? › ./bash_parser ./tests/accept/30-overview.sh

compiler_labs/course main • ? › █
```

Рисунок 16 – Принимаемый сценарий: большой файл

3 РЕЗУЛЬТАТЫ

В ходе курсовой работы сделано следующее:

- Описано учебное подмножество Bash и отдельно указано, какие конструкции полного языка в него не входят.
- На flex реализован лексический анализатор: шобанг, ключевые слова, слова, присваивания, операторы, кавычки и подстановки.
- На bison в режиме yacc реализован синтаксический анализатор. При сборке остаётся один конфликт в правиле local. На ответ программы он не влияет.
- Собран распознаватель bash_parser. Верный текст он принимает без сообщений и завершается с кодом 0. При первой ошибке печатает Error: line N и завершается с кодом 1. Для шобанга, here-документа и условия без [...] к номеру строки добавляется короткое пояснение.

Полный Bash в это подмножество не входит. За его границей остались here-документ, проверка [[...]], арифметика, сложные подстановки, группы команд и запись VAR=value command.

4 ВЫВОД

Цель работы достигнута. Для учебного подмножества Bash составлена собственная контекстно-свободная грамматика и на flex и yacc собран распознаватель. Программа отвечает, принадлежит ли текст подмножеству, и сообщает строку первой ошибки.

Полный язык в одну такую грамматику не укладывается. Here-документ, арифметика и сложные подстановки читаются по своим правилам. Конфликт в local оставлен: на приём и отказ текста он не влияет.

ПРИЛОЖЕНИЕ 1

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "y.tab.h"

extern void error_line(int line, const char *msg);

static int read_single_quote(void);
static int read_double_quote(void);
}%

%option yylineno noyywrap nounput
%x BODY

ID      [A-Za-z_][A-Za-z0-9_]*
WORD    [^ \t\n|&;<>(){}[\]"'`$\\st]+

%%

"#!/bin/bash"[ \t]*\n { BEGIN(BODY); return SHEBANG; }
"#!/bin/bash"[ \t]*   { BEGIN(BODY); return SHEBANG; }
"#!/bin/bash".*       { error_line(1, "extra text in shebang"); exit(1); }
.                     { error_line(1, "missing shebang"); exit(1); }
\n                    { error_line(1, "missing shebang"); exit(1); }
<INITIAL><<EOF>>      { error_line(1, "missing shebang"); exit(1); }

<BODY>[ \tst]+        { }
<BODY>#.st             { }

<BODY>"if"             { return IF; }
<BODY>"then"           { return THEN; }
<BODY>"elif"           { return ELIF; }
<BODY>"else"           { return ELSE; }
<BODY>"fi"             { return FI; }
<BODY>"while"          { return WHILE; }
<BODY>"until"          { return UNTIL; }
<BODY>"do"             { return DO; }
<BODY>"done"           { return DONE; }
<BODY>"for"            { return FOR; }
<BODY>"in"             { return IN; }
<BODY>"case"           { return CASE; }
<BODY>"esac"           { return ESAC; }
<BODY>"function"       { return FUNCTION; }
<BODY>"local"          { return LOCAL; }

<BODY>"2>&1"           { return REDIR_ERR_OUT; }
<BODY>"2>"             { return REDIR_ERR; }
<BODY>">>"             { return DGREAT; }
<BODY>"&&"             { return AND; }
<BODY>"||"             { return OR; }
<BODY>";;"             { return DSEMI; }

<BODY>"|"              { return PIPE; }
<BODY>"&"              { return AMP; }
```

```

<BODY>" ;"                { return SEMI; }
<BODY>"<"                { return LT; }
<BODY>">"                { return GT; }
<BODY>"("                { return LPAREN; }
<BODY>")"                { return RPAREN; }
<BODY> "{"                { return LBRACE; }
<BODY> "}"                { return RBRACE; }
<BODY> "["                { return LBRACK; }
<BODY> "]"                { return RBRACK; }

<BODY>"$("                { return DOLLAR_LPAREN; }
<BODY>"${"{ID}"          { return DOLLAR_BRACE; }
<BODY>"${"                { error_line(yylineno, NULL); exit(1); }
<BODY>"$#"|"${@"|"${*"|"${$"|"${?" { return DOLLAR_SPECIAL; }
<BODY>"${ID}              { return DOLLAR_ID; }
<BODY>"${[0-9]+           { return DOLLAR_NUM; }
<BODY>"$"                { error_line(yylineno, NULL); exit(1); }

<BODY>{ID}={WORD}         { return ASSIGN; }
<BODY>{ID}=               { return ASSIGN; }
<BODY>{ID}                { return ID; }
<BODY>{WORD}              { return WORD; }

<BODY>'                   { return read_single_quote(); }
<BODY>"                   { return read_double_quote(); }

<BODY>\n                  { return NL; }

<BODY>.                   { error_line(yylineno, NULL); exit(1); }

%%

```

```

static int read_single_quote(void)
{
    int line = yylineno;
    int c;

    while ((c = input()) != 0) {
        if (c == '\\')
            return STRING;
        if (c == '\n') {
            error_line(line, NULL);
            exit(1);
        }
    }
    error_line(line, NULL);
    exit(1);
}

```

```

static int read_double_quote(void)
{
    int line = yylineno;
    int c;
    int escaped = 0;

    while ((c = input()) != 0) {

```

```

        if (c == '\n') {
            if (!escaped) {
                error_line(line, NULL);
                exit(1);
            }
            line++;
            escaped = 0;
            continue;
        }
        if (escaped) {
            escaped = 0;
            continue;
        }
        if (c == '\\') {
            escaped = 1;
            continue;
        }
        if (c == '"')
            return STRING;
    }
    error_line(line, NULL);
    exit(1);
}

```

ПРИЛОЖЕНИЕ 2

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
extern int yylineno;
extern FILE *yyin;

void error_line(int line, const char *msg)
{
    if (msg)
        fprintf(stderr, "Error: line %d: %s\n", line, msg);
    else
        fprintf(stderr, "Error: line %d\n", line);
}

void yyerror(char *msg)
{
    (void)msg;
    error_line(yylineno, NULL);
}
%}

%token SHEBANG
%token IF THEN ELIF ELSE FI
%token WHILE UNTIL DO DONE
%token FOR IN CASE ESAC FUNCTION LOCAL
%token ID WORD STRING ASSIGN
%token DOLLAR_ID DOLLAR_NUM DOLLAR_SPECIAL DOLLAR_BRACE DOLLAR_LPAREN
%token LBRACK RBRACK LBRACE RBRACE LPAREN RPAREN
%token PIPE AND OR AMP SEMI DSEMI NL
%token LT GT DGREAT REDIR_ERR REDIR_ERR_OUT

%start program

%%

program:
    SHEBANG list
    ;

list:
    nl_opt
    | nl_opt stmt_seq
    | nl_opt stmt_seq separator
    ;

stmt_seq:
    and_or
    | stmt_seq separator and_or
    ;

separator:
```

```

    SEMI nl_opt
    | AMP nl_opt
    | NL nl_opt
    ;

nl_opt:
    /* empty */
    | nl_opt NL
    ;

and_or:
    pipeline
    | and_or AND nl_opt pipeline
    | and_or OR nl_opt pipeline
    ;

pipeline:
    command
    | pipeline PIPE nl_opt command
    ;

command:
    simple_command
    | assignment
    | test_command
    | if_clause
    | while_clause
    | until_clause
    | for_clause
    | case_clause
    | function_def
    | local_stmt
    ;

assignment:
    ASSIGN
    | ASSIGN word
    ;

simple_command:
    cmd_name arg_list redir_list
    ;

cmd_name:
    ID
    | WORD
    ;

arg_list:
    /* empty */
    | arg_list arg
    ;

arg:
    word
    | ASSIGN

```

```

;

word:
    ID
    | WORD
    | STRING
    | expansion
;

expansion:
    DOLLAR_ID
    | DOLLAR_NUM
    | DOLLAR_SPECIAL
    | DOLLAR_BRACE
    | cmdsubst
;

cmdsubst:
    DOLLAR_LPAREN list RPAREN
;

redir_list:
    /* empty */
    | redir_list redir
;

redir:
    GT redir_target
    | DGREAT redir_target
    | LT redir_target
    | LT LT {
        error_line(yylineno, "here-document is not supported");
        exit(1);
    }
    | REDIR_ERR redir_target
    | REDIR_ERR_OUT
;

redir_target:
    ID
    | WORD
;

test_command:
    LBRACK arg_list RBRACK
;

if_clause:
    IF test_command sep THEN list FI
    | IF test_command sep THEN list elif_parts FI
    | IF test_command sep THEN list ELSE list FI
    | IF test_command sep THEN list elif_parts ELSE list FI
    | IF bad_condition {
        error_line(yylineno, "if condition must be [ ... ]");
        exit(1);
    }

```

```

;

elif_parts:
    ELIF test_command sep THEN list
    | elif_parts ELIF test_command sep THEN list
;

while_clause:
    WHILE test_command sep DO list DONE
    | WHILE bad_condition {
        error_line(yylineno, "while condition must be [ ... ]");
        exit(1);
    }
;

until_clause:
    UNTIL test_command sep DO list DONE
    | UNTIL bad_condition {
        error_line(yylineno, "until condition must be [ ... ]");
        exit(1);
    }
;

/* Условие if/while/until без '[': одно слово и разделитель. */
bad_condition:
    word SEMI
    | word NL
;

for_clause:
    FOR ID IN arg_list sep DO list DONE
;

sep:
    SEMI nl_opt
    | NL nl_opt
;

case_clause:
    CASE arg IN nl_opt case_items ESAC
;

case_items:
    /* empty */
    | case_items case_item
;

case_item:
    patterns RPAREN list DSEMI nl_opt
;

patterns:
    pattern
    | patterns PIPE pattern
;

```

```

pattern:
    ID
    | WORD
    | STRING
    ;

function_def:
    ID LPAREN RPAREN nl_opt brace_group
    | FUNCTION ID nl_opt brace_group
    | FUNCTION ID LPAREN RPAREN nl_opt brace_group
    ;

brace_group:
    LBRACE nl_opt RBRACE
    | LBRACE nl_opt stmt_seq separator RBRACE
    ;

local_stmt:
    LOCAL local_names
    ;

local_names:
    local_item
    | local_names local_item
    ;

local_item:
    ID
    | ASSIGN
    | ASSIGN word
    ;

%%

int main(int argc, char **argv)
{
    int result;

    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            perror(argv[1]);
            return 1;
        }
    }
    result = yyparse();
    if (argc > 1)
        fclose(yyin);
    return result;
}

```